

上板测试指南 — 手势识别系统

面向：(本仓库根目录)，板卡 **PYNQ-Z2** (XC7Z020) 工具：Vivado / Vitis **2025.2** 编写：2026-09-17

本文只写“上板”这一段，且只写能验证的东西。全文按「先排除不可逆的错 → 再逐级放行」排序，每一步都给出判定标准和失败对策。

△ 本文的结论分两类，已分别标注：

- **【已核实】**：来自本机文件/日志，可复查
- **【未验证】**：推理或待实测，**不要当成结论用**

本文用到的四个环境 (**每段代码前都标了在哪跑**)

上板这件事最容易卡住的地方，不是命令本身，而是**不知道该在哪儿敲**。所以本文在**每段代码前面**都加了一个标记：

【终端：PC · Git Bash】	← 在你的开发机上
【终端：PYNQ · Jupyter 或串口】	← 在板子上的 Linux 里
【终端：串口 · MobaXterm】	← 串口终端（看启动日志用）
【终端：仪器 · 万用表】	← 手持仪器，不是电脑

四个环境分别是：

标记	是什么	怎么进去
PC · Git Bash	你的开发机（有 Vivado/Vitis、仓库源码）	右键 → Git Bash Here；或用 MobaXterm 的 Local terminal
PYNQ · Jupyter	板子上的 Python	浏览器开 <a href="http://<板子 IP>:9090">http://<板子 IP>:9090
PYNQ · 串口	板子上的命令行（和 Jupyter 是同一个 Linux）	见 §3.1
仪器	万用表 / 示波器	手持操作，不需要电脑

△ **PC 和 PYNQ 上都有 Python，但装的东西完全不同**：PC 上没有 `pynq` 模块（import pynq 会失败，这是正常的）；PYNQ 上没有 Vivado。在错的那一侧跑，报错是 ModuleNotFoundError 或 command not found —— 看着像环境坏了，其实只是跑错了地方。这就是这些标记存在的理由。`

0. 先说结论：硬件已定，只剩 1 个阻塞

摄像头选型已定：PMOD-CAMERA v1.0 (MUSE LAB)，直插 Pmod A + Pmod B。所以 BD 里的 `clk_wiz_xclk` 和 `io_xclk` 约束都要保留（见 §1.1）。

#	项	状态
B1	摄像头选型	☑已定：PMOD-CAMERA v1.0 (2026-09-17)
B2	<code>io_xclk</code> 引脚约束	☑不缺 —— <code>video_io.xdc:183</code> 有 <code>PACKAGE_PIN Y18</code> （见 §1.2 的更正说明）
B3	<code>rtl/ov5640_regs.v</code> 是占位寄存器表	✗仍是阻塞 —— <code>io_pclk</code> 不会有波形，摄像头不出图
B4	PS7 DDR 参数	☑2026-09-17 已修复并重跑验证（见 §1.4）

现在唯一的硬阻塞是 B3（寄存器表）。

另外还有两件“上电前必做”的事（不是代码问题）：

- 引脚万用表复核 (§2.2) —— PMOD-CAMERA 的原理图是“镜像编号”，XDC 里的映射是推理的，从未实测。插错方向会让 3V3 与 GND 反接。会烧板。
- 短路检查 (§2.1)

§5.2（只跑预处理链）完全不受 B3 影响 —— 它不碰摄像头，现在就可以先做掉这一步，把 PL 的预处理链 + DMA + DDR 通路验通。

1. 前置条件（上电前必须确认的事项）

1.1 ~~B1~~ — 摄像头模块 (**已定: PMOD-CAMERA v1.0** ☒)

2026-09-17 确认：用 MUSE LAB PMOD-CAMERA v1.0，直插 Pmod A + Pmod B。无需转接板、无需焊接。

	值
模块	PMOD-CAMERA v1.0 (TMALL 购入)，公头
接线	J2 → Pmod A (6 根)，J3 → Pmod B (8 根数据)
传感器	OV5640
晶振	无 —— XCLK 必须由 PL 提供 24 MHz
约束	☒ video_io.xdc 第三层就是按它写的，可直接用

⚠为什么这条重要：文档曾经把它写错了

architecture-contract.md §二 曾经整节按 ATK-OV5640 (正点原子) 写，理由是“板载 24 MHz 有源晶振，不需要 PL 提供 XCLK” (2026-09-15 的决策)。那个决策现在被推翻了 —— 回到 PMOD-CAMERA。

这个反复导致了两处连锁错误，都已修正：

- 契约文档 §二/§四 的“14 → 13 根线”、“Arduino 连接器”已改回 Pmod A+B
- 本指南初版据此误判 io_xclk “没有约束” (见 §1.2 的更正)

教训：换模块会连锁影响接线数、XCLK 方向、约束文件、BD 模块四处，每次换都要把这四处重新对一遍，不能只改选型那一句。

两个连接器的信号分配：

J2 → Pmod A (6 根)：XMCLK / DVP_VSYNC / I2C_SDA / DVP_PCLK / DVP_HREF / I2C_SCL
(另有 2 个 NC 脚：ja[3]=Y17, ja[7]=W19)

J3 → Pmod B (8 根)：D6 D4 D2 D0 D7 D5 D3 D1 ← 数据线是**交错**的，不是顺序

⚠Pmod A 与 Raspberry Pi 口 8 根线并联，二者只能用其一 (Y18 Y19 Y16 Y17 U18 U19 W18 W19)。本方案用 Pmod A，RPI 口同时废掉 —— 本项目不用 RPi，可接受。

1.2 ~~B2~~ — `io\_xclk` 约束 (**不缺** ☒)

⚠⚠更正：本指南初版这一节是错的

初版写的是“`io\_xclk` 没有任何引脚约束，线是悬空的”——那是错的。

实际约束在 video_io.xdc:183:

```
set_property -dict { PACKAGE_PIN Y18 IOSTANDARD LVCMOS33 } [get_ports io_xclk]
```

核实方式 (从实现产物回读，不是看文档)：

```
$ grep io_xclk bd_video_wrapper_io_placed.rpt
```

Y18	io_xclk	High Range	I0_L17P_T2_34	OUTPUT	LVCMS033	...	FIXED	...
-----	---------	------------	---------------	--------	----------	-----	-------	-----

Constraint = FIXED —— 约束确实生效了。

初版为什么错：我把“某份规格表里没列 io_xclk”当成了“约束文件里没有它”，**没有做任何验证就写了文档，还把它列成阻塞项。**这正是本指南开头警告的那类错误 —— 只不过这次犯的是我自己。

教训：判断“某个约束在不在”，要 **grep** 约束文件或回读实现报告，**不能凭印象或凭别的文档。**

既然用 PMOD-CAMERA（无晶振），**io_xclk 必须保留：**

项	结论
io_xclk 约束	☑保留 (Y18)
BD 里的 clk_wiz_xclk	☑保留 (100 MHz → 24 MHz, M=12/D=1/O=50, VCO=1200 MHz)
Pmod A 上 U18/W18 等	☑正常排布，没有“白占 IO”的问题

初版还说 `io\_d` 约束了 10 个引脚“越界占了 Pmod A”—— 那也是错的。实际 io_d[7:0] 用的 8 个引脚 (W14 Y14 T11 T10 V16 W16 V12 W13) **正好是 Pmod B 的全部 8 根**，没有越界。当时是被 XDC 里的注释文字（“8bit 数据在 Pmod A”）误导，而代码本身是对的。

唯一真正待确认的是 `Y17` / `W19` (Pmod A 上的 ja[3] / ja[7])：它们是 PMOD-CAMERA 的 NC 脚，因此**没约束是对的**，不用补。

1.3 B3 — 寄存器表必须换成真的

☑2026-09-17 更新：rtl/ov5640_regs.v 已由占位表换为真表（250 条，来源正点原子 i2c_ov5640_rgb565_cfg.v，固化为 640×480 RGB565）。本节由此**不再是硬阻塞** —— 但表内容**未上板实测**，若 io_pclk 仍无波形，要查表内容/接线/时序，不再能一口咬定是占位表的锅。

以下是**更换前**的记录，保留说明当时为什么要换这张表：

rtl/ov5640_regs.v 曾经是**占位表** (rtl/README.md 当时写的：“表里的 8 条寄存器值是猜测，不足以让 OV5640 出图”)。

TB 验的是**接口行为**（拼接顺序、位宽、边界），**不验内容** —— 所以 11/11 PASSED 不代表能用。

症状：万用表/示波器测得 io_xclk 有 24 MHz，但 `io\_pclk` **一直没波形**。看到这个现象先查寄存器表，别去查 RTL。

来源：正点原子例程里的 ov5640cfg.h（或其 Linux 驱动 ov5640.c）。需要的是「RGB565 / 640×480 / 30fps」那套配置。

⚠**别只抄前几条。**OV5640 的配置是**有顺序依赖**的，常见的是 0x3103 软复位 → 0x3008 软复位 → 系统时钟 → 时序 → 输出格式 → 分辨率 → 才到具体画质寄存器。抄漏顺序会静默不出图。

1.4 ~~B4~~ — DDR 参数 (**已修复并验证**)

【已核实】 原 `bd_video.tcl` 里一条 DDR 参数都没有，Vivado 套用了 PS7 出厂默认 `MT41J128M8JP-125`；而 PYNQ-Z2 板载是 `MT41K256M16RE-125`，对不上。

2026-09-17 已在 `bd_video.tcl` 补上三条并加了回读断言：

```
CONFIG.PCW_UIPARAM_DDR_PARTNO      {MT41K256M16 RE-125}  
CONFIG.PCW_UIPARAM_DDR_BUS_WIDTH  {32 Bit}  
CONFIG.PCW_UIPARAM_DDR_FREQ_MHZ   {533.333}
```

2026-09-17 21:53 已重跑验证 —— 脚本输出：

```
>>> DDR 断言通过 (MT41K256M16 RE-125)
```

产物已更新（比特流 + XSA 里都是正确配置）。**这一条不再是阻塞项。**

将来若重建工程，判定标准仍然是日志里出现 `>>> DDR 断言通过 (MT41K256M16 RE-125)`。**没有这一行就是没生效**，别继续。

2. 不接电的安全检查

2.1 短路检查

万用表**电阻档**（或蜂鸣档），测 3V3 对 GND：

读数	含义
鸣一声 / $< 10\ \Omega$	短路，绝对不要上电
几百 Ω 以上 / 逐渐上升	正常（电容充电）

对摄像头模块单独测一次，再对插上之后的板子测一次。

▼ 现场记录（打印后手填）

测点	读数	判定（ $< 10\ \Omega$ 短路过 / 数百 Ω 正常）

2.2 引脚万用表复核（**烧板风险，不能跳**）

【未验证】XDC 的引脚映射是推理的，没实测过。

只用量电平和通断，不用示波器，也不需要上电：

【终端：仪器·万用表（通断档）】

- ① 万用表 → 通断档
- ② 表笔黑端接 PYNQ-Z2 的 GND（挑一个板上标了 GND 的排针）
- ③ 表笔红端逐个点 Pmod A 的 8 个物理针
- ④ 通的那个就是 GND 列 — 记下来
- ⑤ 同样方法找 3V3（接 3.3V 排针）
- ⑥ 对照 video_io.xdc 末尾的预期表，核对 GND/3V3 是否落在同一列

判定：如果 GND/3V3 的位置和 video_io.xdc 里推的不一致，**立刻停止，回来改 XDC**，不要“先插上试试”。

这一步花 5 分钟，省掉一块板子。

▼ 现场记录（打印后手填）

Pmod 脚位	信号	XDC 预期引脚	实测（通/断）	与预期一致？
---------	----	----------	---------	--------

3. 上电与 PS 侧验证 (**先不加载比特流**)

3.1 上电顺序

【终端：硬件 · 断电状态下插线，最后才上电】

- ① 插好 SD 卡 (PYNQ 镜像, v3.0.1)
- ② 接串口线 (Micro USB, 板上标 UART)
- ③ 打开串口终端 (115200 8N1)
- ④ 最后插 12V 电源

串口终端选哪个 —— 推荐 MobaXterm (家庭版, 免费) :

工具	评价
MobaXterm Home Edition ☒ 首选	串口 + SSH + SFTP 三合一, 而且免费、有免安装便携版。 本项目后面要做的三件事它一个顶三个: · 串口看 PYNQ 启动日志 (Session → Serial, 选 COM 口 + 115200) · SSH 登板子跑命令 (串口只能看, 敲长命令不方便) · SFTP 拖拽传 .bit / .hwh (省掉记 scp 命令)
PuTTY	够用且极小, 但只有串口/SSH, 没有文件传输 —— 传 .bit 还得另找 WinSCP。只当串口终端可以
tio / picocom	Linux/macOS 上的命令行串口工具, 简洁可靠。 本机是 Windows, 用不上
各类"串口助手"小工具	△只收发热门波特率的能用, 但看不到 Linux 启动全过程的滚屏, 排查"卡在 Loading kernel"时很吃亏

△波特率 115200 / 8 数据位 / 无校验 / 1 停止位 (8N1), 流控关掉。这是 PYNQ 镜像的默认值, 改了会看到乱码。

△没串口就等于没有眼睛 —— PYNQ 起没起来、卡在哪一步, 全看串口输出。先把这个弄通再往下, 不要靠"板子上灯亮了"猜。

▼ 现场记录 (打印后手填)

项	设置 / 观察	是否正常

3.2 光一个指示灯就能砍掉一半问题

观察	含义
板上 DONE 灯亮	比特流加载成功
只有 PWR 灯亮	卡在 FSBL / 没加载
串口有 PYNQ 启动日志到 login:	Linux 起来了, DDR 正常
串口卡在 Loading kernel... 或乱码	多半是 DDR 参数问题 → 回 §1.4

这一步就能验证 B4。DDR 错了, Linux 根本起不来。

3.3 进 PYNQ

【终端：串口 • MobaXterm (或 PuTTY) • 115200 8N1】

```
# 串口里
login: xilinx
password: xilinx

# 看网口 IP (PC 与板子同网段)
ifconfig
```

浏览器开 <http://<板子 IP>:9090> → Jupyter。或者直接用串口跑 Python。

4. 加载 overlay

4.0 先把两个文件弄出来 (**XSA 里就有, 不用重新跑**)

PYNQ 要的是 `.bit` + `.hwh` 两个文件, 它们都在仓库里的 `gesture_system.xsa` 中 —— `.xsa` 本质是个 zip, 直接解压即可, 不需要重跑综合:

【终端: PC • Git Bash】

```
# 在 PC 上, 仓库根目录
cd vivado/gesture_system
unzip -o gesture_system.xsa -d /tmp/xsa_extract

# 看看里面有什么
unzip -l gesture_system.xsa | grep -E "\.bit|\.hwh"
#   bd_video.hwh           744,025 B
#   gesture_system.bit     4,045,692 B    ← 与 build-report 记录的字节数一致, 可据此核对
```

⚠ `.hwh` 在 xsa 里叫 `bd_video.hwh`, 但必须改名为 `gesture_system.hwh` —— PYNQ 靠同名配对找 IP 表: `gesture_system.bit` ↔ `gesture_system.hwh`。名字不一致时 PYNQ 不会报“找不到 hwh”, 而是只认出 `default` 一个 IP, 然后你在 `ip_dict` 里一个我们的 IP 都看不到 —— 很容易误判成 overlay 有问题。

【终端: PC • Git Bash】

```
# 传到板子 (改成同名同目录)
scp gesture_system.bit xilinx@<板子 IP>:/home/xilinx/
scp /tmp/xsa_extract/bd_video.hwh xilinx@<板子 IP>:/home/xilinx/gesture_system.hwh
```

核对拿对了没有:

文件	大小应为	对不上说明
<code>gesture_system.bit</code>	4,045,692 字节	不是这个数 → xsa 是旧的, 重跑 <code>create_project.tcl</code>
<code>gesture_system.hwh</code>	744,025 字节	同上

两个文件必须同名同目录 (PYNQ 靠配对的 `.hwh` 解析 IP 表)。

【终端: PYNQ • Jupyter (浏览器:9090) 或串口】

```
from pynq import Overlay
ol = Overlay("/home/xilinx/gesture_system.bit")

# ⚠ 先看它到底认出了什么, 别猜
print(sorted(ol.ip_dict.keys()))
```

期望看到 (【已核实】来自 `bd_video.hwh`) :

```
vdma, v_tc, dma_in, dma_out, gesture_preproc_0
```

立刻记下地址 (后面每一步都要用) :

```
for n in ["vdma", "v_tc", "dma_in", "dma_out", "gesture_preproc_0"]:
    ip = ol.ip_dict.get(n)
    print(f"{n:20s} {ip}")
```

【已核实】.hwh 里的地址是这个（不是 docs/architecture-contract.md §2.14 写的 0x4300_0000/0x43C0_0000，那是原 Sobel 工程的残留）：

IP	BASE
gesture_preproc_0	0x4000_0000
dma_in	0x41E0_0000
dma_out	0x41E1_0000
v_tc	0x44A0_0000
vdma	0x44A1_0000

⚠ 别把这些地址硬编码进驱动。项目自己的规则就是 "地址由 PL 侧驱动分配后写入寄存器告知，不硬编码"（契约 §3.1）。上面这张表只用于对账：如果 ip_dict 报出来的和这里不一样，说明 overlay 不对。

版本警告：PYNQ 镜像与 Vivado 2025.2 的兼容性【未验证】。报版本错时：

```
ol = Overlay("gesture_system.bit", ignore_version=True)
```

▼ 现场记录（打印后手填）

文件	理论大小	实际大小	一致?

5. 验证顺序 (**这是全文最重要的一节**)

按顺序走。**每一步失败就停下**，不要跳过去试下一步 —— 后面所有现象都会被前面没验证的环节污染。

- | | | |
|--------------------------|-----------|----------|
| ① 加载 overlay, ip_dict 认全 | → 本文 §4 | |
| ② DDR 能分配 buffer, 读写一致 | → 本文 §5.1 | ← 不需要摄像头 |
| ③ 只跑预处理链 (不碰摄像头) | → 本文 §5.2 | ← 不需要摄像头 |
| ④ io_xclk 有 24 MHz | → 本文 §5.3 | |
| ⑤ io_scl 有脉冲 (SCCB 在跑) | → 本文 §5.3 | |
| ⑥ io_pclk 有波形 (摄像头配置成功) | → 本文 §5.3 | |
| ⑦ io_href / io_vsync 有脉冲 | → 本文 §5.3 | |
| ⑧ 数据线 io_d 不是恒 0 | → 本文 §5.3 | |
| ⑨ VDMA 能抓到帧 | → 本文 §5.4 | ← 依赖 ⑥⑦⑧ |
| ⑩ 全链路: 摄像头 → 96x96 | → 本文 §5.5 | ← 依赖 ⑨+⑩ |

②③ 不需要摄像头，所以摄像头还没到货也能先做掉 —— 现在就可以做。

☑②③ 已脚本化: `host/bringup\_check.py`

上面 ②③ 两步的代码不用手敲了 —— 已做成可直接跑脚本:

```
python3 host/bringup_check.py          # 跑 ②③
python3 host/bringup_check.py --step 2  # 只跑 DDR 自检
python3 host/bringup_check.py --json r.json # 结果落盘 (报告用)
```

判定: ***** BRINGUP CHECK PASSED *****。它会在每一步给出**针对性的排查提示** (比如输出全黑时列出三个常见原因)。

⚠④~⑧ 需要示波器/逻辑分析仪，脚本不做 —— 那些是模拟信号的逐级排查，没法自动化。但 ②③ 脚本化之后，摄像头插上时只剩一条线要查。

注意本文的 § 编号和别的文档会撞车 (比如 `architecture-contract.md` 也有 §5)。下文凡是引用项目内的其他文档，一律写成 `文件名 §x.y`；只写 `§x.y` 的都是指**本指南**。引用其他文档的地方是：

写在哪	指的是
§1.1 表格「出处」列	<code>hardware-checklist.md</code> 第三章、 <code>architecture-contract.md</code> 第二章
§4 表格上方的注	<code>architecture-contract.md</code> §2.14 (那里写的地址 已过时 ，勿用)
§4 引用契约	<code>architecture-contract.md</code> §3.1
§5.2 启动顺序	<code>docs/architecture-contract.md</code> §4.3
§5.3 上方	<code>docs/hardware-checklist.md</code> 第二章 (工具采购建议)
§6.1 表格「出处」列	<code>skill 09-pitfalls.md</code> 的 C1/C3/E1、 <code>gui-reproduction-guide.md</code> §2.9

5.1 DDR 与 buffer 基本自检

【终端：PYNQ • Jupyter 或串口】

```
from pynq import allocate
import numpy as np

buf = allocate(shape=(9216,), dtype=np.uint8)
buf[:] = np.arange(9216, dtype=np.uint8)
print("读回一致:", np.array_equal(buf, np.arange(9216, dtype=np.uint8)))
print("物理地址:", hex(buf.physical_address))
```

判定：打印 **True** 且有非 0 物理地址。 **失败** → 回 §1.4, DDR 参数不对。

5.2 第一步真正的功能验证：**只跑预处理链**（不需要摄像头）

这是**当前最应该先做的事**—— 它把摄像头这个最大的不确定性排除在外。

数据流：

```
DDR(输入 640×480 RGB565) → dma_in(MM2S) → gesture_preproc → dma_out(S2MM) → DDR(96×96)
```

⚠️**启动顺序不能反** ([docs/architecture-contract.md §4.3](#)) :

- ① 填输入数据 + flush cache
- ② 先武装 dma_out (S2MM) ← 必须先, 否则预处理输出的第一拍没有接收方
- ③ 再启动 dma_in (MM2S)
- ④ 最后 ap_start
- ⑤ 轮询 ap_done
- ⑥ invalidate cache, 读结果

【终端：PYNQ • Jupyter 或串口】

```
inbuf = allocate(shape=(640*480,), dtype=np.uint16) # RGB565
outbuf = allocate(shape=(9216,), dtype=np.uint8)    # 96×96

# 用 host/dump_frame.py 造的测试图当输入, 或先填个已知图案
# ... 填 inbuf ...

inbuf.flush()
outbuf.flush()

# ⚠️ 顺序: 先 S2MM, 再 MM2S
dma_out.recvchannel.transfer(outbuf)
dma_in.sendchannel.transfer(inbuf)

gp = ol.gesture_preproc_0
gp.write(0x00, 0x01) # ⚠️ AP_START 在 0x00, 不是 0x04

while not (gp.read(0x00) >> 1) & 1: # ⚠️ bit1 = AP_DONE, 也在 0x00
    pass

outbuf.invalidate()
```

△△ 状态位在 `0x00`，不在 `0x04`

这是本项目踩过的大坑 (skill 09-pitfalls.md D2)：HLS 生成的寄存器里根本没有名为 STATUS 的寄存器。

偏移	名称	作用
0x00	CTRL	控制 + 状态位就在这里
0x04	GIER	全局中断使能 —— 不是状态!
0x08	IP_IER	通道中断使能
0x0C	IP_ISR	通道中断状态

CTRL 位域: bit0=AP_START(RW) · bit1=AP_DONE(RO) · bit2=AP_IDLE(RO) · bit3=AP_READY(RO)

别自己给寄存器起名。从 .hwh 提取真实偏移:

```
python <skill>/pynq/ip_contract.py gesture_system.hwh --ip gesture_preproc_0
```

判定: outbuf 里的值和 PC 上 golden 对得上。

【终端: PC • Git Bash】

```
# PC 侧
python host/dump_frame.py stats board.bin
python host/dump_frame.py compare board.bin golden.bin
python host/dump_frame.py show board.bin --png out.png
```

这一步跑通, 说明 PL 的预处理链 + 两次 DMA + DDR 通路全部正常。后面摄像头出问题就一定能在采集侧。

5.3 摄像头信号逐级排查 (有了示波器/逻辑分析仪再做**)**

△没有示波器或逻辑分析仪的话, 这一节做不了。docs/hardware-checklist.md §二自己写了: "没有它, 只能靠猜和改代码, 调试周期会从半天变成一周"。这是本项目最值得买的一个 ¥40 工具。

按顺序量, 每一级失败都给出确定的结论:

步骤	测哪里	期望	没有的话说明
①	io_xclk	24 MHz 方波	PL 侧问题, 与摄像头无关。查 Clocking Wizard (约束在 §1.2 已确认存在)
②	io_scl	有脉冲 (配置期间)	sccb_master 没在跑。查 rst_n、时钟
③	io_pclk	有波形	摄像头没配上 → 查 ov5640_regs.v 表内容 (§1.3) 和 sccb_master.cfg_error
④	io_href / io_vsync	有脉冲	配上但没出图 → 查寄存器表的输出格式/分辨率

⑤	io_d[7:0]	不是恒 0/恒 1	数据线接错或采集时序问题
---	-----------	-----------	--------------

①②③ 的划分是本节的价值所在：

- ① 没有 → **PL 的问题**，摄像头完全是无辜的
- ① 有、② 有、③ 没有 → **SCCB 配置失败**，是寄存器表或 SCCB 时序
- ③ 有、④⑤ 没有 → 摄像头在工作，**问题在时序或数据线**

▼ 现场记录 (打印后手填)

级	测点	期望	实测 (频率/有无)	判定

5.4 VDMA 抓帧

前提： §5.3 的 ⑥⑦⑧ 都通过 (有 PCLK/HREF/VSYNC 和有效数据) 。

VDMA 是 **3 帧缓存** (`c_num_fstores=3`)，地址不硬编码：

【终端：PYNQ • Jupyter 或串口】

```
vdma = ol.vdma
# 地址从 hwh/ip_dict 读，不要写死 0x44A10000
```

判定： 把 DDR 里 VDMA 写的帧 dump 出来，用 `dump_frame.py show` 看到**摄像头画面** (不是全黑、不是噪声) 。

▼ 现场记录 (打印后手填)

项	实测值	说明

--	--	--

5.5 全链路

§5.4 通过 + §5.2 通过 → 直接跑 `host/gesture_overlay.py`:

【终端：PYNQ • Jupyter 或串口】

```
from gesture_overlay import GesturePipeline
g = GesturePipeline()
g.print_info()          # 先看 IP 认出来没有
g.setup_dma()
g.config()
g.fill_test_pattern()   # 无摄像头时验数据通路
g.run_once()
g.show()                # Jupyter 里出图
```

结果与 PC golden 对拍：

【终端：PC • Git Bash】

```
python host/dump_frame.py stats board.bin
python host/dump_frame.py compare board.bin golden.bin
```

数据格式：裸 `.bin` 无头, `uint8`, 行优先, $96 \times 96 = 9216$ 字节。

`stats` 子命令会打 **4×4 分块均值** —— 这一项能区分 "整体偏移"和"只有一块有内容 (ROI 位置问题)"。

▼ 现场记录 (打印后手填)

项	结果	备注

6. 已知会误导你的现象 (**读这一节省几小时**)

6.1 工具流程全过，不代表能用

本项目的记录里，至少三类问题在综合/实现/比特流阶段全都静默通过：

现象	真实原因	出处
比特流正常生成	DMA 的 <code>M_AXI_S2MM</code> 根本没连到 PS 的 HP 口	skill 09-pitfalls.md C3
比特流正常生成	AXI 互连的 <code>ARESETN</code> 悬空 → 被 tie-off 到 0 → 事务一条都过不去	GUI 复现指南 §2.9
时序 WNS 是正确的	<code>apply_board_preset</code> 把 PS7 配置覆盖了	skill 09-pitfalls.md C1

所以：不要用“工具跑通了”当作通过标准。用 §5 的分级验证。

6.2 两条容易误判的报错

报错	判定
<code>[Common 17-1257] Failed to create directory 'C' / [Common 17-354] Could not open 'C' for writing</code>	△对产物无害，但对流程致命——它会让 <code>wait_on_run</code> 抛错、脚本中断。 不要当成纯噪声 。已在 <code>create_project.tcl</code> 里加了带 <code>reset_run</code> 的重试（详见 §7.1）
<code>[Netlist 29-160] Cannot set property 'iostandard' ... for objects of type 'pin' x100</code>	忽略。PS7 IP 自生成 XDC 的作用域约束在顶层无处落地。DDR 约束同样内建在 <code>.dcp</code> 里，仍然生效。已在 <code>create_project.tcl</code> 里降为 INFO

判断一条报错要不要管的通用方法：只看 `ERROR` 和 `CRITICAL WARNING` 两种级别，再用 ``grep -c "^ERROR" impl_1/runme.log`` 数一下。一个 33KB 的日志里有 111 条 `WARNING` 但 0 条 `ERROR` 是正常的（skill 09-pitfalls.md E1）。

6.3 ``runme.bat` / `runme.sh` 改了没用`

它们是 Vivado 每个 run 每次重新生成的。想改 run 行为要用 `config_*` 参数或 `pre-hook`，改这两个文件下次就被覆盖。

6.4 想复用旧工程目录时

`create_project.tcl` 默认删掉整个工程目录重建。如果 GUI 里改过东西、不想被抹掉，加 `--keep`：

【终端：PC • Git Bash】

```
vivado -mode batch -source create_project.tcl -tclargs --keep
```

但注意：`--keep` 不会让 BD 重新生成，所以 §1.4 的 DDR 修改不会生效。改过 bd_video.tcl 就必须不带 --keep` 重跑。`

7. 附：本次排查留下的技术结论

7.1 `Common 17-1257 / 17-354` —— 对产物无害，**对流程致命**

【已核实】 只出现在两个 IP 的 OOC 综合日志，且都在同一次启动的同一时刻：

```
runme.log:15 ERROR: [Common 17-1257] Failed to create directory 'C'.
runme.log:16 source bd_video_dma_in_0.tcl -notrace          ← 它跑到这，没退出
runme.log:17 create_project: Time (s): cpu = 00:00:11      ← 正常返回
```

证据链（产物层面确实无害）：

1. `source` 在第 3 行就执行了，报错在第 15 行 —— 说明它发生在 `run` 脚本之外
2. 返回码 0，脚本继续跑完，末尾 `synth_design completed successfully`
3. 产物齐全（`.dcp / _sim_netlist.v / _stub.v / __synthesis_is_complete__`）
4. 每次运行只有 2-3 个 OOC run 中招，中招的也照样完成了

原因：一次性启动 16-22 个 OOC 综合，每个 Vivado 实例都要建一批临时目录——**并发建目录竞争 / 杀软-文件索引拦截**的瞬时失败。重跑即成功。

已排除：`TEMP/TMP` 正常（**系统临时目录**）；没有空环境变量会被展开成裸盘符；工程脚本里没有任何地方传过 `"C"`。

△△更正（2026-09-17，实测推翻初版结论）

初版本指南写的是**"100% 无害，直接忽略"**——那是错的。

它对**产物**无害，但对**流程**致命：

```
ERROR: [Vivado 12-13638] Failed runs(s) : 'bd_video_ic_ctrl_imp_xbar_0_synth_1'
ERROR: [Common 17-39] 'wait_on_runs' failed due to earlier errors.  ← 脚本在此中断
```

Vivado 会把这个瞬时失败判定成 **run 失败**，于是 `wait_on_run` 直接抛错返回，脚本中断——**XSA 导出和资源报告根本没执行到**。2026-09-17 实际发生过：比特流成功生成了，但没有 `.xsa`。

初版为什么误判：当时的依据是“产物齐全 + 返回码 0”，**没有考虑它对上层批处理流程的影响**。教训：一个错误“是不是无害”，要看**谁在消费它的后果**，不能只看直接产物。

现在的处理（`create_project.tcl` 里）：

```
proc run_with_retry {run_name launch_args {max_attempts 3}} {
    for {set attempt 1} {$attempt <= $max_attempts} {incr attempt} {
        if {$attempt > 1} { catch {reset_run -quiet $run_name} } ;# ← 关键
        catch {launch_runs $run_name -jobs 8 {*}$launch_args}
        catch {wait_on_run $run_name}
        ...
    }
}
```

△`reset\_run` 是必需的，漏了就变成“假重试”：`launch_runs` 对已经失败的 `run` 不做任何事（它认为“跑过了”）。实测日志：

```
>>> synth_1 进度 0% — 第 1 次未完成
>>> 重试...
[21:38:34] Waiting for synth_1 to finish...
```

```
[21:38:34] synth_1 finished      ← 同一秒返回，根本没跑
>>> synth_1 进度 0% — 第 2 次未完成
```

先 `reset_run` 清掉失败状态，`launch_runs` 才会真正执行。（已完成的子 run 不受影响，只有失败的需重跑。）

7.2 `Netlist 29-160` —— 无害，但掩盖了一个真问题

【已核实】

- 报错行 `bd_video_ps7_0.xdc:29` 是 `set_property iostandard "SSTL15_T_DCI" [get_ports "DDR_VRP"]`
- 这份 XDC 是 **PS7 IP 自动生成的** (`set_property PIO_DIRECTION` 是 PS7 专有属性)
- OOC 综合阶段 **0 报错** (`bd_video_ps7_0_synth_1 / dma_in / dma_out` 都查过)
- 只在顶层 `link_design` 读 `.dcp` 时炸 —— 顶层 36 个 user IO 里 *没有一个是 `DDR`***，约束无处落地
- 顶层 35 个 logical port 中 22 个是 `hdmi_vid_out_*` (本来就悬空、走 DRC 豁免)
- DDR 引脚/电平约束**同时内建在 `.dcp` 里**，仍然生效
- 那次实现 **ERROR 0 条**，比特流正常生成

但真正的问题是：这 100 条告警刷屏，把 \$1.4 那个 DDR 参数错误盖住了。`ps7_parameters.xml` 里写着 `MT41J128M8 JP-125` (默认值)，而 `bd_video.tcl` 里根本没有 DDR 配置 —— 一条告警都没提这件事。

7.3 已完成的修改 (2026-09-17)

文件	改动
<code>vivado/bd_video.tcl</code>	补 PS7 DDR 三条参数 + 回读断言 (写错就早失败，不静默走默认值)
<code>vivado/create_project.tcl</code>	加 <code>set_msg_config -id {Netlist 29-160} -new_severity INFO</code>

△ 两处修改都**只改了脚本**。现有比特流 (2026-09-16 21:25) 里是旧配置，必须按 \$1.4 重跑才能生效。

8. 待验证清单（按"说清楚什么没验证"的原则）

项	状态
板上时序余量	△RTL 级 WNS = +0.265 ns、WHS = +0.051 ns (2026-09-17 最后一次实现, All user specified timing constraints are met)。△逐次波动大 (+0.873 / +1.177 / +0.265, 布线是随机的)。△△且该 WNS 属于 AMD `v_tc` IP 内部, 不是本设计的余量 (见 report/design.md §4.5); 本项目 HLS 流水线余量 +43%。板级实测仍未做
PS7 DDR 参数	☑已修复并重跑验证 (§1.4), XSA 里是 MT41K256M16 RE-125
摄像头模块选型	☑已定: PMOD-CAMERA v1.0, 直插 Pmod A+B (2026-09-17, §1.1)
io_xclk 引脚约束	☑存在 (video_io.xdc:183 = Y18, 实现报告 Constraint=FIXED)。初版误判为"缺失"已更正 (§1.2)
`ov5640_regs.v` 寄存器表	☑2026-09-17 已换为真表 (250 条, 固化 640×480)。△未上板实测
引脚映射 (万用表复核)	✗未做 (§2.2)
PYNQ 镜像与 2025.2 兼容性	✗未验证
HDMI 输出	✗未实现 (BD 里没有 TMDS 编码器; 22 个端口在比特流里悬空, 上板不要接 HDMI 线)
板上实测	✗完全未做 (板子未到货)
建工程→综合→实现→比特流→XSA 全流程	☑2026-09-17 完整跑通 (0 error / 0 critical warning, 顶层 bd_video_wrapper)

9. 一页速查

【终端：PC • Git Bash】

```
# ---- 建工程 + 出比特流 (改了 bd_video.tcl 之后必须重跑) ----
cd <仓库根>/vivado
vivado -mode batch -source create_project.tcl
# 只看 DDR 断言过没过:
# >>> DDR 断言通过 (MT41K256M16 RE-125)

# ---- 只检查 BD 是否合法 (约 1 分钟, 不跑综合) ----
vivado -mode batch -source create_project.tcl -tclargs --synth 0

# ---- 没板子也能跑的自检 ----
bash rtl/run_iverilog.sh          # 判定: *** TB PASSED *** (finished 不算)
bash sw/build_preproc_sim.sh      # 判定: *** PREPROC DRIVER SIM PASSED ***

# ---- PC 侧数据处理 ----
python host/dump_frame.py stats board.bin
python host/dump_frame.py compare board.bin golden.bin
python host/dump_frame.py show board.bin --side-by-side golden.bin --png cmp.png
```

【终端：PYNQ • Jupyter 或串口】

```
# ---- 板上 PYNQ ----
from pynq import Overlay, allocate
ol = Overlay("gesture_system.bit", ignore_version=True)
print(sorted(ol.ip_dict.keys()))
# 期望: vdma, v_tc, dma_in, dma_out, gesture_preproc_0
```

验证顺序一句话:

先跑 §5.2 (预处理链, 不要摄像头) → 通了再碰摄像头
摄像头按 io_xclk → io_scl → io_pclk → HREF/VSYNC → 数据 逐级量

三个"看到就停"的信号:

看到	说明
串口卡在 Loading kernel...	DDR 参数不对 → §1.4
io_xclk 没波形	PL 问题, 别去动摄像头
io_pclk 没波形但 XCLK 有	查寄存器表内容 / SCCB 接线 / 时序 → §1.3

10. 打印版 (Word 操作手册)

本文还有一份**排版好的 Word 版**，适合打印出来带到工位上：

【终端：PC • Git Bash (或直接看仓库里的文件)】

```
docs/上板测试操作手册.docx
```

它是从本文自动生成的，不是手工维护的第二份。改了本文件之后重跑一次即可：

【终端：PC • Git Bash】

```
python scripts/build_manual.py      # 生成 docx
python scripts/test_manual.py      # 校验没有内容在渲染中丢失
```

⚠ **不要直接改 Word**。Word 版是渲染产物，手工改的内容下次重新生成就没了，而且两份会开始分叉——这个项目已经在“双目录分叉”上吃过一次亏。**要改内容就改本文件（`.md`）**，它是**唯一事实来源**。

Word 版针对打印做的处理：

项	说明
版面	A4、页边距 2cm、正文 10.5pt
分页	每个二级标题另起一页 ——手上拿一张做一个阶段
代码块	等宽 + 灰底 + 左边框（终端命令抄错一个字符就白跑一轮）
引用块	左边框高亮（本手册的“⚠注意”全在引用块里）
页眉页脚	文档名 + 第 X 页 / 共 Y 页
手填记录表	万用表读数、示波器频率这类 当场要记的数 ， 每个相关小节末尾都有一张空白表

⚠ **不生成目录 (TOC)**。python-docx 生成的是“域”，Word 打开时要手动按 F9 更新才显示页码——打印前忘了更新会印出一页空白。与其埋这个坑，不如自己在 Word 里插。页脚的页码同理，打开后 Ctrl+A → F9 更新一次即可。

— 手册结束 —